Cryptographie R309
Chiffrement RSA

12 octobre 2026

Objectif Le but de cette séance est d'implémenter le chiffrement RSA de bout en bout : génération des clés, découpage d'un message en blocs, chiffrement et déchiffrement. Les outils arithmétiques de la séance précédente (`PGCD`, `inv_mod_fast`, `puissancemodulaire`, `test_primalite`) sont réutilisés.

Rendu `rsa.py`

Le code devra être réalisé sous `Python`. Les fonctions demandées (en rose dans l'énoncé) devront être sauvegardées dans un fichier nommé `rsa.py`, à déposer sur le Moodle dans la boîte prévue. Le fichier ne doit contenir ni variable globale ni test. Il doit être *autonome* : recopiez-y les fonctions nécessaires du TP précédent (en bleu dans l'énoncé).

Test `test-rsa.py`

Un script de test est fourni afin de valider votre code au fur et à mesure du TP. Les fonctions aléatoires ne sont pas comparées à une valeur fixe : le script vérifie les propriétés de ce qu'elles renvoient.

Rappel Une clé RSA est formée de $n = pq$, où p et q sont premiers, et de deux exposants e et d tels que

$$ed \equiv 1\ [\varphi(n)], \qquad \varphi(n) = (p-1)(q-1).$$

Le couple (e, n) est la clé publique, (d, n) la clé privée. Pour tout $0 \leq m < n$, on a $(m^e)^d \equiv (m^d)^e \equiv m\ [n]$: les deux exposants s'inversent mutuellement. Dans ce TP on chiffre avec d et on déchiffre avec e : c'est la convention de la *signature*, où seul le détenteur de la clé privée peut produire le message, mais tout le monde peut le lire. Pour la confidentialité, on échange simplement les rôles.

Exercice 1 Construction des clés

1. *(Tirage d'un nombre premier)* Écrire une fonction `random_prime` qui prend un entier N et renvoie un nombre « probablement » premier p de N bits, c'est-à-dire
$$2^{N-1} \leq p < 2^N. \tag{1}$$
On tirera des entiers au hasard dans cet intervalle jusqu'à en trouver un qui passe `test_primalite` avec 10 tests de Fermat. Combien de tirages faut-il en moyenne pour $N = 128$? *Indication : la proportion de nombres premiers autour de x est d'environ $1/\ln x$.*

2. *(Génération des clés)* Écrire une fonction `random_key` qui prend un entier N et renvoie le triplet (e, d, n). Les nombres p et q seront tirés avec `random_prime`(N) (et distincts). L'exposant e sera tiré au hasard entre 65537 et $\varphi(n) - 1$, et retiré tant qu'il n'est pas premier avec $\varphi(n)$; son inverse d sera calculé avec `inv_mod_fast`. Pour des raisons de sécurité (attaque de Wiener), on renouvellera le tirage de e tant que $d \leq n^{1/4}/3$. On notera que d'après (1),
$$2^{2(N-1)} \leq n = pq < 2^{2N}, \tag{2}$$
donc n compte $2N - 1$ ou $2N$ bits.

Exercice 2 Découpage du message en blocs

RSA chiffre des entiers plus petits que n. Un message doit donc être converti en une suite d'entiers de moins de $2N-1$ bits : on le découpe en blocs de $2(N-1)$ bits, ce qui garantit d'après (2) que chaque bloc est strictement inférieur à n.

1. *(Découpage)* Écrire une fonction `string2tab` qui prend une chaîne de caractères m et un entier N et renvoie la liste des entiers obtenus ainsi : on code m en UTF-8, on écrit chaque octet sur 8 bits, on ajoute à la fin un bit 1 puis autant de bits 0 que nécessaire pour atteindre une longueur multiple de $2(N-1)$, et on découpe en blocs de $2(N-1)$ bits, chacun converti en entier. Le bit 1 ajouté sert de *marqueur de fin* : il permettra de retirer le bourrage sans ambiguïté.
Pour convertir une chaîne en une suite de bits, on pourra utiliser
```
''.join(f'{i:08b}' for i in m.encode())
```
Par exemple, le message `Hello` donne les 40 bits
```
0100100001100101011011000110110001101111
```
puis, après le marqueur, 41 bits :

```
010010000110010101101100011011000110111111
```

Pour $N = 3$, les blocs font $2(N-1) = 4$ bits ; on ajoute 3 zéros pour atteindre 44 bits :

```
0100 1000 0110 0101 0110 1100 0110 1100 0110 1111 1000
```

et l'on obtient la liste

```
[4, 8, 6, 5, 6, 12, 6, 12, 6, 15, 8]
```

La conversion d'une chaîne `s` de 0 et de 1 en entier s'écrit `int(s, 2)`.

Remarque. On peut écrire un code plus efficace en manipulant directement des entiers et des décalages de bits plutôt que des chaînes de 0 et 1. Ce n'est pas demandé.

2. *(Reconstitution)* Écrire la fonction inverse `tab2string`, qui prend une liste d'entiers et N et reconstitue la chaîne de caractères. Chaque entier est réécrit sur exactement $2(N-1)$ bits (`f'{x:0{L}b}'` avec `L` la largeur), les bits sont concaténés, puis on supprime le bourrage : tout ce qui suit le *dernier* bit 1, et ce bit lui-même. Pour convertir une chaîne de bits `b` en texte :

```
int(b, 2).to_bytes((len(b) + 7) // 8, byteorder='big').decode('utf-8')
```

Vérifier l'aller-retour sur `Hello` avec $N = 3$, puis sur un message contenant des accents. Pourquoi le marqueur de fin est-il indispensable ? Que se passerait-il sans lui pour un message terminé par un caractère dont le code se finit par des zéros ?

Exercice 3 Chiffrement et déchiffrement

1. *(Chiffrement)* Écrire une fonction `chiffrement_rsa` qui prend un message (chaîne de caractères) et une clé d, n, et renvoie la liste des entiers

$$x_i = m_i^d \bmod n,$$

où m_i est le i-ème entier produit par `string2tab`. On utilisera `puissancemodulaire`.

L'entier N qui fixe la taille des blocs n'est pas un argument : on le retrouve à partir de n. D'après (2), si ℓ est le nombre de bits de n (`n.bit_length()`), alors $\ell \in \{2N-1, 2N\}$, donc

$$N \leq \frac{\ell+1}{2} < N+1,$$

et N est la partie entière de $(\ell+1)/2$.

2. *(Déchiffrement)* Écrire une fonction `dechiffrement_rsa` qui prend une liste d'entiers et une clé e, n, et renvoie le message obtenu en appliquant `tab2string` à la liste des

$$y_i = x_i^e \bmod n.$$

Vérifier l'aller-retour sur un message de votre choix avec une clé de $N = 64$ puis $N = 512$.

3. *(subsidiaire)* Chiffrer deux fois le même message avec la même clé. Qu'observe-t-on ? En quoi est-ce une faiblesse ? *(On parle de chiffrement déterministe ; les implémentations réelles ajoutent de l'aléa au bourrage.)*

Quelques méthodes utiles en Python. `random.randrange(a, b)` ; `n.bit_length()` ; `str.rfind` ; `f'{x:08b}'` ; `int(s, 2)` ; `bytes.decode`.